Green University of Bangladesh

Department of Computer Science and Engineering (CSE)

Faculty of Sciences and Engineering

Semester: (Summer, Year: 2026), B.Sc. in CSE (Day)

Open-Ended Laboratory Report

Course Title: Artificial Intelligence Lab

Course Code: CSE-316 **Section:** 241-D2

Project / Experiment Title: Smart Pac-Man Intelligent Enemy Navigation Using A-star and Minimax

Student Details

No.	Name	Student ID
1	Md. Reahoon Zannah	223002038
2	MD. Aminul Islam	223002004
3	Tawabur Rahman Sakib	222902058

Open-Ended Lab Date : 17.08.2026

Submission Date : 17.08.2026

Course Teacher's Name : Sian Ashsad

Open-Ended Laboratory Assessment Status	
Marks: _____	**Signature:** _____
Comments: _____	**Date:** _____

1 Title

Smart Pac-Man Intelligent Enemy Navigation Using A-star and Minimax

2 Objectives and Problem Statement

2.1 Objectives

- To calculate the shortest path to the player using the A* search algorithm for Ghost navigation.
- To predict and intercept Pac-Man's escape routes through a Minimax decision model.
- To optimize real-time performance and reduce computational overhead using Alpha-Beta pruning.
- To display the A* path and Minimax decision nodes on the screen for clear visualization.

2.2 Problem Statement

In classic Pac-Man style games, enemy (ghost) movement is often based on simple, non-intelligent rules, which makes the enemy either too weak or too predictable. This project addresses that problem by building an intelligent enemy-navigation system for a grid-based maze game. The **input** to the system is the current maze layout (a 15×15 grid of walls and walkable cells), the ghost's current position, and the player's (Pac-Man's) current position. The system uses **A* Search** to compute the shortest walkable path from the ghost to the player, and uses **Minimax with Alpha-Beta Pruning** to look ahead multiple future moves, treating the ghost as the maximizing agent and the player as the minimizing (escaping) agent. The **output** is the ghost's next optimal move on the grid. The expected result is that the ghost consistently narrows the distance to the player, corners it efficiently, and eventually catches the player faster and more intelligently than a simple greedy or random-walk enemy would.

3 Proposed Solution Design

3.1 Relevant Concepts and Problem Analysis

The AI concepts applied in this project are the **A* Search Algorithm**, the **Minimax Algorithm**, and **Alpha-Beta Pruning**. The maze is represented as a 15×15 grid, where each cell is either a wall (1) or a walkable path (0). This grid is treated as a graph in which each walkable cell is a node connected to its up/down/left/right walkable neighbours. A* operates directly on this grid graph to find the shortest path from the ghost to the player, while Minimax operates on a game tree built from the same grid, where each level of the tree alternates between the ghost's possible moves (maximizing) and the player's possible moves (minimizing).

3.2 Selected Method and Justification

Two AI techniques were combined for the ghost's decision-making:

A* Search is used to compute $f(n) = g(n) + h(n)$, where $g(n)$ is the path cost from the ghost's start to the current node and $h(n)$ is the Manhattan-distance heuristic to the player's position. A* is suitable here because it guarantees the shortest path in an unweighted grid while exploring far fewer nodes than an uninformed search such as BFS, which keeps the ghost responsive in real time.

Minimax with Alpha-Beta Pruning is used on top of A* so that the ghost does not simply chase the player's current position but reasons a few moves ahead: the ghost (MAX) tries to minimize the distance to the player and reduce the player's escape options, while the player (MIN) is simulated as trying to maximize its own escape options. Alpha-Beta Pruning is used to cut off branches of this game tree that cannot possibly change the final decision, which reduces the number of nodes evaluated and keeps the search depth ($MINIMAX\_DEPTH = 3$) computable within real time (60 FPS).

3.3 Main Solution Steps

1. Represent the maze as a 2D grid and build a `Maze` class that reports walkable neighbours for any cell.
2. Initialize the player (Pac-Man) and the ghost (enemy) at fixed starting positions, and populate coins on every remaining walkable cell.
3. On every ghost update tick, run **A*** from the ghost's position to the player's current position to obtain the shortest path and the resulting path cost.
4. Run **Minimax with Alpha-Beta Pruning** up to a fixed depth to evaluate future ghost/-player move sequences, using an evaluation function based on Manhattan distance and the player's escape options.
5. Select the ghost move recommended by Minimax and advance the ghost one cell toward that move.
6. Update the score, collect coins, and check the game state (player caught, all coins collected, or game continues).
7. Render the maze, the A* path, the Minimax-evaluated nodes, the Alpha-Beta pruned branches, and the live statistics panel (nodes evaluated, alpha/beta bounds, execution time).

3.4 Algorithm / Pseudocode / Flowchart

Listing 1: Pseudocode: A* + Minimax with Alpha-Beta Pruning

```
function A_STAR(maze, start, goal):
    frontier <- priority queue containing start, priority 0
    cost[start] <- 0
    while frontier is not empty:
        current <- node in frontier with lowest f(n)
        if current == goal: break
        for each neighbor of current:
            new_cost <- cost[current] + 1
            if neighbor not in cost or new_cost < cost[neighbor]:
                cost[neighbor] <- new_cost
                f(neighbor) <- new_cost + manhattan(neighbor, goal)
```

```
                add neighbor to frontier with priority f(neighbor)
    return reconstructed shortest path from start to goal

function MINIMAX(ghostPos, playerPos, depth, maximizing, alpha, beta):
    if ghostPos == playerPos: return WIN_SCORE
    if depth == 0: return EVALUATE(ghostPos, playerPos)
    if maximizing: // Ghost's turn
        best <- -infinity
        for each move in neighbors(ghostPos):
            score <- MINIMAX(move, playerPos, depth-1, False, alpha, beta)
            best <- max(best, score)
            alpha <- max(alpha, best)
            if beta <= alpha: break // Alpha-Beta Pruning
        return best
    else: // Player's simulated turn
        best <- +infinity
        for each move in neighbors(playerPos):
            score <- MINIMAX(ghostPos, move, depth-1, True, alpha, beta)
            best <- min(best, score)
            beta <- min(beta, best)
            if beta <= alpha: break // Alpha-Beta Pruning
        return best
```

4 Implementation and Technical Details

The project is implemented in **Python 3** using the **Pygame** library for the graphical interface, grid rendering, and real-time input handling. No external AI libraries were used; the A* search, Minimax, and Alpha-Beta Pruning algorithms were implemented from scratch using Python's built-in `heapq` module for the A* priority queue. The maze, player, coins, and ghost each have their own class (`Maze`, `Player`, `CoinManager`, `GhostAI`) to keep the search logic separate from rendering. The most important part of the implementation is the `GhostAI` class, which combines A* (for path awareness) with Minimax and Alpha-Beta Pruning (for lookahead decision-making).

Listing 2: Minimax with Alpha-Beta Pruning (core AI decision function)

```
def minimax(self, maze, ghost_pos, player_pos, depth, maximizing, alpha, beta):
    self.stats.nodes += 1
    if ghost_pos == player_pos:
        return 10000 + depth * 100
    if depth == 0:
        return self.evaluate_state(maze, ghost_pos, player_pos)

    if maximizing:
        # MAX = Ghost. It chooses the highest-scoring chase position.
        best = -math.inf
        moves = maze.neighbors(ghost_pos)
        for index, nxt in enumerate(moves):
            score = self.minimax(maze, nxt, player_pos, depth - 1, False, alpha,
                beta)
            best = max(best, score)
            alpha = max(alpha, best)
```

```
            if beta <= alpha:
                self.pruned.extend(moves[index + 1:])
                break
        return best

    # MIN = Pac-Man. It selects the simulated move best for escaping.
    best = math.inf
    moves = maze.neighbors(player_pos)
    for index, nxt in enumerate(moves):
        score = self.minimax(maze, ghost_pos, nxt, depth - 1, True, alpha, beta
            )
        best = min(best, score)
        beta = min(beta, best)
        if beta <= alpha:
            break
    return best
```

Code Explanation:

The function above is a recursive Minimax search with Alpha-Beta Pruning. When `maximizing` is true, it represents the ghost's turn: the ghost tries every possible move and keeps the one with the highest score, updating `alpha` to the best value found so far. When `maximizing` is false, it represents the player's simulated turn: the player tries every possible move and keeps the one with the lowest score (best escape), updating `beta`. Whenever `beta <= alpha`, the remaining sibling branches are skipped (pruned) because they can no longer influence the final decision at a higher level of the tree. At `depth == 0`, the recursion stops and `evaluate_state()` scores the position using the A* distance to the player and the player's number of escape routes.

5 Result Analysis and Discussion

5.1 Output Evidence

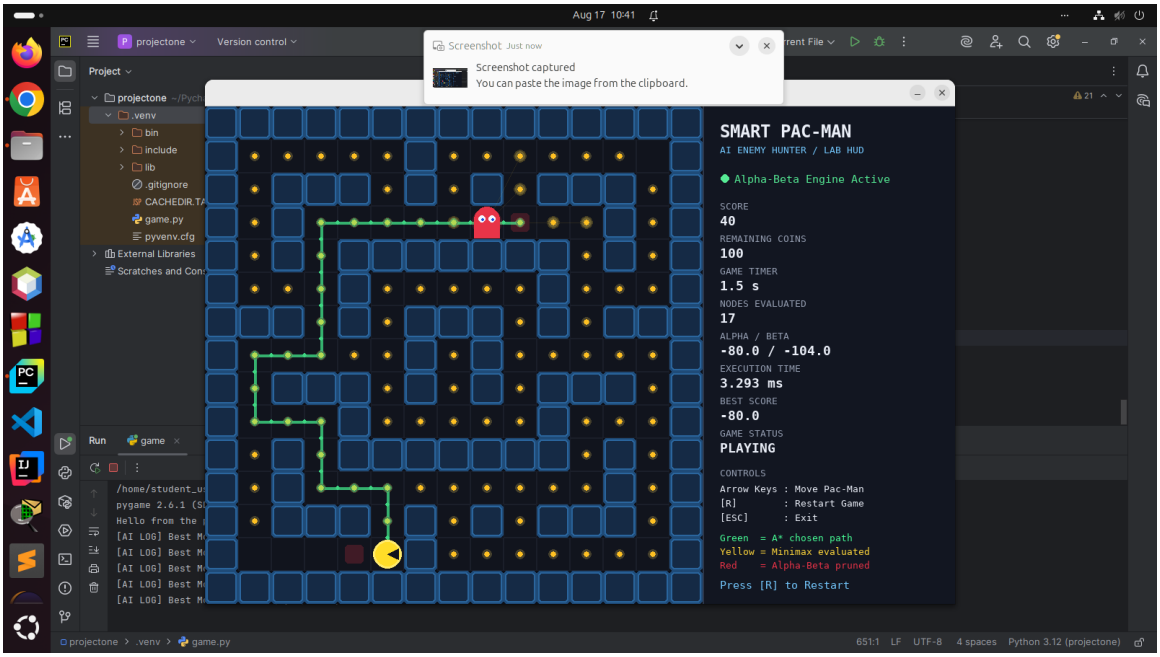

Figure 1: Smart Pac-Man gameplay starting

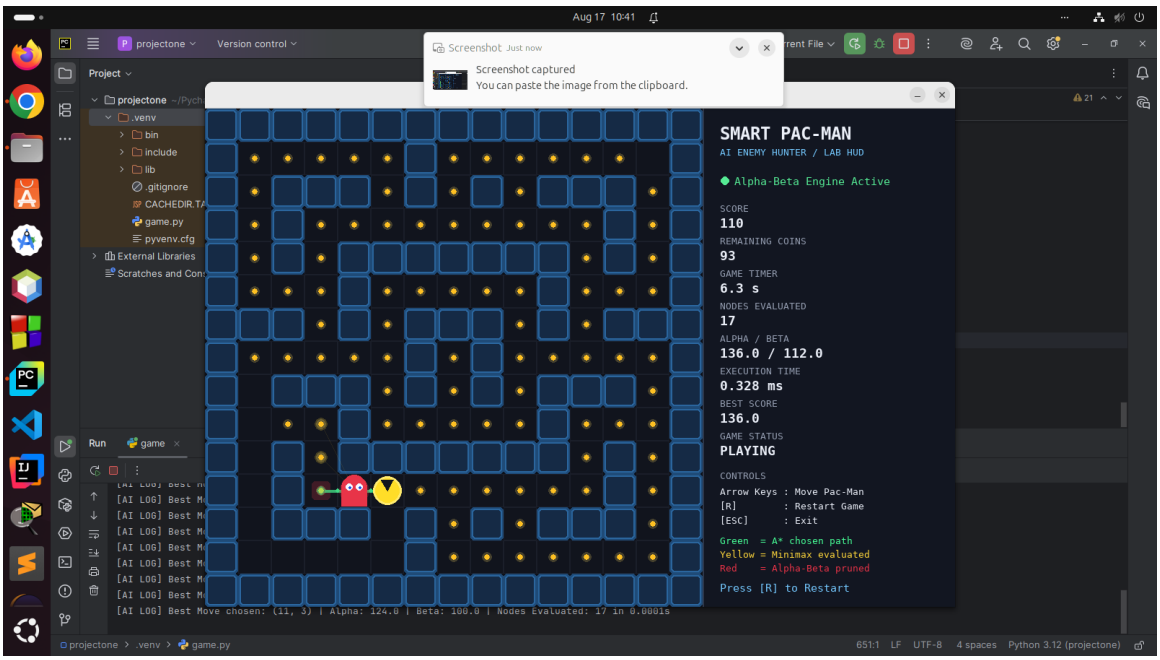

Figure 2: Smart Pac-Man gameplay running

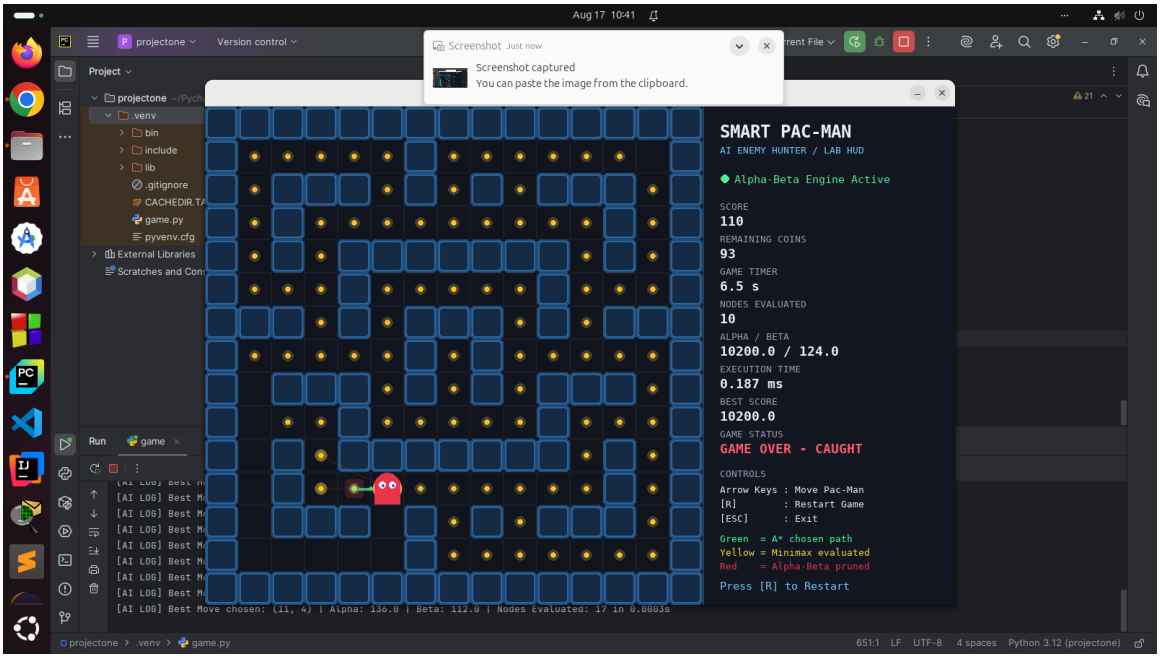

Figure 3: Smart Pac-Man loose

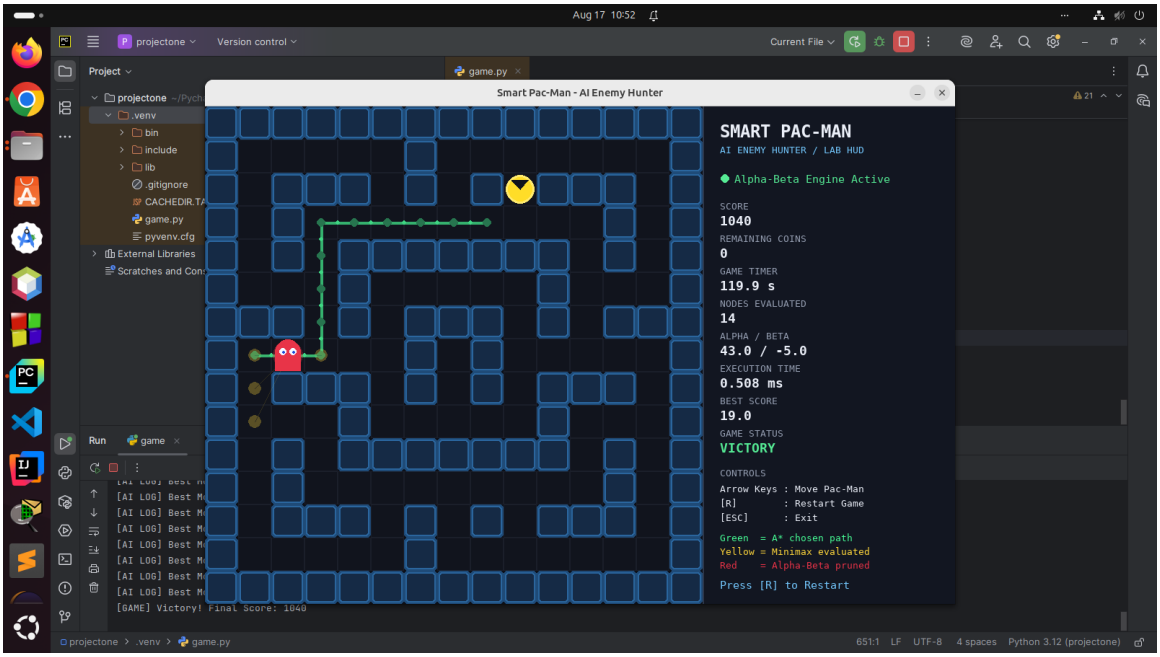

Figure 4: Smart Pac-Man victory

5.2 Analysis, Observation, and Challenges

The program produced the expected result: the ghost consistently computed a valid shortest path with A* and used Minimax with Alpha-Beta Pruning to anticipate the player's escape moves rather than simply following the shortest path blindly. One relevant observation is the number of **nodes evaluated** versus **nodes pruned** at search depth 3 – Alpha-Beta Pruning noticeably reduced the number of Minimax branches explored compared to plain Minimax, which kept the execution time low enough (a few milliseconds per decision) to run smoothly at 60 FPS. One difficulty faced during implementation was balancing the ghost's move delay (`GHOST_DELAY`) against the Minimax search depth: a higher depth produced smarter, more

anticipatory movement but increased computation time per decision. This was solved by tuning `MINIMAX_DEPTH = 3` and `GHOST_DELAY = 0.210` seconds so the AI stays responsive while still reasoning several moves ahead.

6 Conclusion, Limitations and Future Improvement

6.1 Conclusion

This project successfully implemented an intelligent enemy-navigation system for a Pac-Man style maze game by combining A* Search for shortest-path computation with Minimax and Alpha-Beta Pruning for strategic lookahead. The resulting ghost AI is able to pursue the player efficiently, corner it using predicted escape routes, and adapt its decisions in real time, demonstrating how classical search and adversarial game-tree algorithms can be combined to produce believable intelligent agent behaviour in a grid-based game environment.

6.2 Limitations and Future Improvement

- **Depth Limitation:** Currently, the Minimax depth is fixed at 3. Increasing the depth further causes exponential growth in nodes, which, even with pruning, can start to impact framerates in more complex maze layouts.
- **Future Improvement - Iterative Deepening:** Implementing Iterative Deepening Search alongside Minimax would allow the AI to search as deep as possible within a strict time limit per frame, rather than a fixed depth.
- **Future Improvement - Multi-Agent Coordination:** Introducing multiple ghosts that use a shared communication model to coordinate traps (e.g., one chases, one cuts off) rather than independent calculation.

7 References

1. Artificial Intelligence Lab (CSE-316) Laboratory Manual, Department of CSE, Green University of Bangladesh.
2. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020. (A* Search, Minimax, and Alpha-Beta Pruning.)
3. Pygame Documentation, `https://www.pygame.org/docs/`.

8 Individual Contributions

Table 1: Individual contributions

No.	Student Name and ID	Specific Contribution
1	Md. Reahoon Zannah (223002038)	Latex Report, Coding and Github
2	MD. Aminul Islam (223002004)	Latex finetune and testing
3	Tawabur Rahman Sakib (222902058)	Project Code and algorithm

A Complete Source Code

GitHub Repository: Smart Pac-Man game – AI Lab Open-Ended Lab

Listing 3: Complete source code

```
1
2  """
3  SMART PAC-MAN: AI Enemy Hunter
4  University AI Lab Assignment - Single-file Python 3 + Pygame project.
5  Install: pip install pygame
6  Run: python main.py
7  """
8  import heapq
9  import math
10 import time
11 from dataclasses import dataclass
12 import pygame
13 # ============================================================================
14 # SECTION 1: CONFIGS, GRID MAPS, AND COLOR PALETTES
15 # ============================================================================
16 ROWS, COLS = 15, 15
17 CELL = 46
18 MAZE_W, MAZE_H = COLS * CELL, ROWS * CELL
19 SIDEBAR_W = 350
20 WIDTH, HEIGHT = MAZE_W + SIDEBAR_W, MAZE_H
21 FPS = 60
22 PLAYER_DELAY = 0.105
23 GHOST_DELAY = 0.210
24 MINIMAX_DEPTH = 3
25 # 1 = metallic wall, 0 = walkable path
26 MAZE_MAP = [
27     "111111111111111",
28     "100000100000001",
29     "101110101011101",
30     "101000000000101",
31     "101011111110101",
32     "100010000010001",
33     "111010111010111",
34     "100000101000001",
35     "101110101011101",
36     "100010000010001",
37     "101011111110101",
38     "101000000000101",
39     "101110101011101",
40     "100000100000001",
41     "111111111111111",
42 ]
43 PLAYER_START = (13, 1)
44 GHOST_START = (1, 13)
45 BG = (13, 16, 24)
46 PATH = (17, 20, 29)
47 GRID = (27, 31, 42)
48 PANEL = (18, 22, 33)
```

```
PANEL_BORDER = (45, 55, 74)
WALL = (21, 42, 69)
WALL_EDGE = (43, 102, 158)
WALL_GLOW = (31, 75, 120)
YELLOW = (255, 221, 40)
YELLOW_GLOW = (255, 235, 110)
RED = (232, 52, 72)
WHITE = (245, 248, 255)
BLUE = (30, 80, 220)
GOLD = (255, 193, 35)
GOLD_GLOW = (255, 222, 105)
GREEN = (72, 255, 151)
SEARCH = (255, 215, 60)
TEXT = (235, 239, 248)
MUTED = (150, 159, 178)
ACCENT = (117, 201, 255)
DANGER = (255, 92, 110)
SUCCESS = (86, 235, 149)
DIRECTIONS = {
    "UP": (-1, 0), "DOWN": (1, 0),
    "LEFT": (0, -1), "RIGHT": (0, 1),
}
# ============================================================================
# SECTION 2: GRID REPRESENTATION & MAZE BUILDER
# ============================================================================
class Maze:
    """Static maze plus helper methods used by movement and AI algorithms."""
    def __init__(self, source):
        self.grid = [[int(value) for value in row] for row in source]
    def is_walkable(self, row, col):
        inside = 0 <= row < ROWS and 0 <= col < COLS
        return inside and self.grid[row][col] == 0
    def neighbors(self, position):
        row, col = position
        result = []
        for dr, dc in DIRECTIONS.values():
            nr, nc = row + dr, col + dc
            if self.is_walkable(nr, nc):
                result.append((nr, nc))
        return result
    def walkable_cells(self):
        result = []
        for row in range(ROWS):
            for col in range(COLS):
                if self.is_walkable(row, col):
                    result.append((row, col))
        return result
def cell_center(position):
    row, col = position
    return col * CELL + CELL // 2, row * CELL + CELL // 2
def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
# ============================================================================
# SECTION 3: PLAYER & COLLECTIBLE COIN CLASSES
```

```
# ================================================================================
@dataclass
class Coin:
    row: int
    col: int
    collected: bool = False
    @property
    def position(self):
        return self.row, self.col
class Player:
    """Pac-Man moves one grid cell at a time using the arrow keys."""
    def __init__(self):
        self.reset()
    def reset(self):
        self.position = PLAYER_START
        self.direction = "RIGHT"
        self.last_move = 0.0
    def move(self, maze, direction, now):
        if now - self.last_move < PLAYER_DELAY:
            return False
        dr, dc = DIRECTIONS[direction]
        row, col = self.position
        destination = (row + dr, col + dc)
        if not maze.is_walkable(*destination):
            return False
        self.position = destination
        self.direction = direction
        self.last_move = now
        return True
class CoinManager:
    """Places collectible coins on every normal walkable maze cell."""
    def __init__(self, maze):
        self.coins = []
        for row, col in maze.walkable_cells():
            if (row, col) not in (PLAYER_START, GHOST_START):
                self.coins.append(Coin(row, col))
    def reset(self):
        for coin in self.coins:
            coin.collected = False
    def collect(self, position):
        for coin in self.coins:
            if coin.position == position and not coin.collected:
                coin.collected = True
                return True
        return False
    def remaining(self):
        return sum(1 for coin in self.coins if not coin.collected)
# ================================================================================
# SECTION 4: AI ENGINE (A* ALGORITHM & MINIMAX WITH ALPHA-BETA PRUNING)
# ================================================================================
@dataclass
class AIStats:
    nodes: int = 0
    alpha: float = -math.inf
```

```
    beta: float = math.inf
    execution_ms: float = 0.0
    best_score: float = 0.0
class GhostAI:
    """
    A* supplies shortest-path knowledge.
    Minimax predicts player escape moves.
    Alpha-Beta Pruning skips branches that cannot improve the decision.
    """
    def __init__(self):
        self.reset()
    def reset(self):
        self.position = GHOST_START
        self.last_move = time.perf_counter()
        self.path = []
        self.evaluated = []
        self.pruned = []
        self.stats = AIStats()
    def a_star(self, maze, start, goal):
        """Classic A*: f(n) = g(n) + Manhattan heuristic h(n)."""
        if start == goal:
            return [start]
        frontier = [(0, start)]
        came_from = {start: None}
        cost = {start: 0}
        while frontier:
            _, current = heapq.heappop(frontier)
            if current == goal:
                break
            for nxt in maze.neighbors(current):
                new_cost = cost[current] + 1
                if nxt not in cost or new_cost < cost[nxt]:
                    cost[nxt] = new_cost
                    priority = new_cost + manhattan(nxt, goal)
                    heapq.heappush(frontier, (priority, nxt))
                    came_from[nxt] = current
        if goal not in came_from:
            return []
        path = []
        current = goal
        while current is not None:
            path.append(current)
            current = came_from[current]
        path.reverse()
        return path
    def evaluate_state(self, maze, ghost_pos, player_pos):
        """Higher is better for the ghost."""
        path = self.a_star(maze, ghost_pos, player_pos)
        distance = len(path) - 1 if path else 999
        escape_options = len(maze.neighbors(player_pos))
        distance_score = 130 - distance * 12
        trap_bonus = (4 - escape_options) * 9
        return distance_score + trap_bonus
    def minimax(self, maze, ghost_pos, player_pos, depth, maximizing, alpha,
```

```
        beta):
211         self.stats.nodes += 1
212         if ghost_pos not in self.evaluated:
213             self.evaluated.append(ghost_pos)
214         if ghost_pos == player_pos:
215             return 10000 + depth * 100
216         if depth == 0:
217             return self.evaluate_state(maze, ghost_pos, player_pos)
218         if maximizing:
219             # MAX = Ghost. It chooses the highest-scoring chase position.
220             best = -math.inf
221             moves = maze.neighbors(ghost_pos)
222             for index, nxt in enumerate(moves):
223                 score = self.minimax(
224                     maze, nxt, player_pos, depth - 1, False, alpha, beta
225                 )
226                 best = max(best, score)
227                 alpha = max(alpha, best)
228                 self.stats.alpha = max(self.stats.alpha, alpha)
229                 # beta <= alpha means remaining siblings cannot change the
                        choice.
230                 if beta <= alpha:
231                     self.pruned.extend(moves[index + 1:])
232                     break
233             return best
234         # MIN = Pac-Man. It selects the simulated move best for escaping.
235         best = math.inf
236         moves = maze.neighbors(player_pos)
237         if not moves:
238             return self.evaluate_state(maze, ghost_pos, player_pos)
239         for index, nxt in enumerate(moves):
240             score = self.minimax(
241                 maze, ghost_pos, nxt, depth - 1, True, alpha, beta
242             )
243             best = min(best, score)
244             beta = min(beta, best)
245             self.stats.beta = min(self.stats.beta, beta)
246             if beta <= alpha:
247                 self.pruned.extend(moves[index + 1:])
248                 break
249         return best
250     def choose_move(self, maze, player_pos):
251         start = time.perf_counter()
252         self.stats = AIStats()
253         self.evaluated = []
254         self.pruned = []
255         legal_moves = maze.neighbors(self.position)
256         if not legal_moves:
257             self.path = []
258             return self.position
259         best_move = legal_moves[0]
260         best_score = -math.inf
261         alpha = -math.inf
262         beta = math.inf
```

```
        for move in legal_moves:
            if move not in self.evaluated:
                self.evaluated.append(move)
            score = self.minimax(
                maze, move, player_pos,
                MINIMAX_DEPTH - 1, False, alpha, beta
            )
            if score > best_score:
                best_score = score
                best_move = move
            alpha = max(alpha, best_score)
            self.stats.alpha = max(self.stats.alpha, alpha)
        # The complete A* route chosen by the tactical decision is visualized.
        route = self.a_star(maze, best_move, player_pos)
        self.path = [self.position] + route if route else []
        self.stats.best_score = best_score
        self.stats.execution_ms = (time.perf_counter() - start) * 1000.0
        print(
            f"[AI LOG] Best Move chosen: {best_move} | "
            f"Alpha: {self.bound(self.stats.alpha)} | "
            f"Beta: {self.bound(self.stats.beta)} | "
            f"Nodes Evaluated: {self.stats.nodes} "
            f"in {self.stats.execution_ms / 1000.0:.4f}s"
        )
        return best_move
    def update(self, maze, player_pos, now):
        if now - self.last_move < GHOST_DELAY:
            return False
        self.position = self.choose_move(maze, player_pos)
        self.last_move = now
        return True
    @staticmethod
    def bound(value):
        if value == math.inf:
            return "+INF"
        if value == -math.inf:
            return "-INF"
        return f"{value:.1f}"
# ============================================================================
# SECTION 5: PYGAME GRAPHICS ENGINE & HUD RENDERING (DRAW FUNCTIONS)
# ============================================================================
class Renderer:
    """Responsible only for drawing. Game logic stays outside this class."""
    def __init__(self, screen):
        self.screen = screen
        self.title_font = pygame.font.SysFont("consolas", 24, bold=True)
        self.heading_font = pygame.font.SysFont("consolas", 18, bold=True)
        self.font = pygame.font.SysFont("consolas", 15)
        self.small = pygame.font.SysFont("consolas", 13)
    def draw(self, maze, player, ghost, coins, score, seconds, status):
        self.screen.fill(BG)
        self.draw_maze(maze)
        self.draw_search_overlay(ghost, seconds)
        self.draw_coins(coins, seconds)
```

```
317         self.draw_astar_path(ghost)
318         self.draw_pacman(player, seconds)
319         self.draw_ghost(ghost, seconds)
320         self.draw_hud(ghost, score, coins.remaining(), seconds, status)
321     def draw_maze(self, maze):
322         for row in range(ROWS):
323             for col in range(COLS):
324                 rect = pygame.Rect(col * CELL, row * CELL, CELL, CELL)
325                 if maze.grid[row][col] == 1:
326                     pygame.draw.rect(self.screen, WALL_GLOW, rect, border_radius
                            =5)
327                     inner = rect.inflate(-4, -4)
328                     pygame.draw.rect(self.screen, WALL, inner, border_radius=5)
329                     pygame.draw.rect(
330                         self.screen, WALL_EDGE, inner, width=2, border_radius=5
331                     )
332                 else:
333                     pygame.draw.rect(self.screen, PATH, rect)
334                     pygame.draw.rect(self.screen, GRID, rect, width=1)
335     def draw_coins(self, coin_manager, seconds):
336         pulse = 1.0 + math.sin(seconds * 6.0) * 0.18
337         for coin in coin_manager.coins:
338             if coin.collected:
339                 continue
340             center = cell_center(coin.position)
341             glow = pygame.Surface((18, 18), pygame.SRCALPHA)
342             pygame.draw.circle(
343                 glow, (*GOLD_GLOW, 55), (9, 9), max(5, int(7 * pulse))
344             )
345             self.screen.blit(glow, (center[0] - 9, center[1] - 9))
346             pygame.draw.circle(self.screen, GOLD, center, 4)
347     def draw_pacman(self, player, seconds):
348         x, y = cell_center(player.position)
349         radius = CELL // 2 - 6
350         pygame.draw.circle(self.screen, YELLOW_GLOW, (x, y), radius + 3)
351         pygame.draw.circle(self.screen, YELLOW, (x, y), radius)
352         mouth = math.radians(18 + abs(math.sin(seconds * 8.0)) * 27)
353         facing = {"RIGHT": 0, "DOWN": 90, "LEFT": 180, "UP": 270}
354         angle = math.radians(facing[player.direction])
355 
356         p1 = (x + math.cos(angle - mouth) * radius,
357               y + math.sin(angle - mouth) * radius)
358         p2 = (x + math.cos(angle + mouth) * radius,
359               y + math.sin(angle + mouth) * radius)
360         pygame.draw.polygon(self.screen, PATH, [(x, y), p1, p2])
361 
362     def draw_ghost(self, ghost, seconds):
363         x, y = cell_center(ghost.position)
364         body_w = CELL - 10
365         radius = body_w // 2
366 
367         glow = pygame.Surface((CELL, CELL), pygame.SRCALPHA)
368         glow_alpha = int(45 + abs(math.sin(seconds * 4.0)) * 40)
369         pygame.draw.circle(
```

```
            glow, (*RED, glow_alpha), (CELL // 2, CELL // 2), CELL // 2 - 2
        )
        self.screen.blit(glow, (x - CELL // 2, y - CELL // 2))

        pygame.draw.circle(self.screen, RED, (x, y - 4), radius)
        pygame.draw.rect(
            self.screen, RED, (x - radius, y - 4, body_w, radius + 8)
        )
        foot_radius = max(3, body_w // 8)
        for index in range(4):
            foot_x = x - radius + foot_radius + index * foot_radius * 2
            pygame.draw.circle(self.screen, RED, (foot_x, y + radius),
                foot_radius)

        for offset in (-7, 7):
            eye = (x + offset, y - 5)
            pygame.draw.circle(self.screen, WHITE, eye, 5)
            pygame.draw.circle(self.screen, BLUE, (eye[0] + 1, eye[1] + 1), 2)

    def draw_astar_path(self, ghost):
        if len(ghost.path) < 2:
            return

        points = [cell_center(position) for position in ghost.path]
        overlay = pygame.Surface((MAZE_W, MAZE_H), pygame.SRCALPHA)
        pygame.draw.lines(overlay, (*GREEN, 170), False, points, width=4)

        for point in points:
            pygame.draw.circle(overlay, (*GREEN, 105), point, 6)

        # Small arrows communicate path direction to an examiner.
        for index in range(len(points) - 1):
            x1, y1 = points[index]
            x2, y2 = points[index + 1]
            middle = ((x1 + x2) // 2, (y1 + y2) // 2)
            pygame.draw.circle(overlay, (*GREEN, 210), middle, 3)

        self.screen.blit(overlay, (0, 0))

    def draw_search_overlay(self, ghost, seconds):
        """Yellow = visited Minimax nodes, red = Alpha-Beta pruned siblings."""
        overlay = pygame.Surface((MAZE_W, MAZE_H), pygame.SRCALPHA)
        pulse = 0.55 + abs(math.sin(seconds * 8.0)) * 0.45
        yellow_alpha = int(80 * pulse)

        evaluated = list(dict.fromkeys(ghost.evaluated))[:28]
        pruned = list(dict.fromkeys(ghost.pruned))[:20]
        ghost_center = cell_center(ghost.position)

        for cell in evaluated:
            center = cell_center(cell)
            pygame.draw.circle(
                overlay, (*SEARCH, yellow_alpha), center, 9
            )
```

```
            pygame.draw.line(
                overlay, (*SEARCH, int(yellow_alpha * 0.45)),
                ghost_center, center, width=1
            )

        for row, col in pruned:
            rect = pygame.Rect(
                col * CELL + 10, row * CELL + 10, CELL - 20, CELL - 20
            )
            pygame.draw.rect(overlay, (*RED, 55), rect, border_radius=5)

        self.screen.blit(overlay, (0, 0))

    def stat(self, x, y, label, value, value_color=TEXT):
        self.screen.blit(self.small.render(label, True, MUTED), (x, y))
        self.screen.blit(
            self.heading_font.render(value, True, value_color), (x, y + 18)
        )

    def draw_hud(self, ghost, score, remaining, seconds, status):
        panel_rect = pygame.Rect(MAZE_W, 0, SIDEBAR_W, HEIGHT)
        pygame.draw.rect(self.screen, PANEL, panel_rect)
        pygame.draw.line(
            self.screen, PANEL_BORDER, (MAZE_W, 0), (MAZE_W, HEIGHT), width=2
        )

        x, y = MAZE_W + 24, 20
        self.screen.blit(
            self.title_font.render("SMART PAC-MAN", True, TEXT), (x, y)
        )
        self.screen.blit(
            self.small.render("AI ENEMY HUNTER / LAB HUD", True, ACCENT),
            (x, y + 32),
        )
        y += 72

        pygame.draw.circle(self.screen, SUCCESS, (x + 7, y + 8), 6)
        self.screen.blit(
            self.font.render("Alpha-Beta Engine Active", True, SUCCESS),
            (x + 20, y),
        )
        y += 38

        stats = [
            ("SCORE", str(score)),
            ("REMAINING COINS", str(remaining)),
            ("GAME TIMER", f"{seconds:.1f} s"),
            ("NODES EVALUATED", str(ghost.stats.nodes)),
            ("ALPHA / BETA",
             f"{ghost.bound(ghost.stats.alpha)} / {ghost.bound(ghost.stats.beta)
                 }"),
            ("EXECUTION TIME", f"{ghost.stats.execution_ms:.3f} ms"),
            ("BEST SCORE", f"{ghost.stats.best_score:.1f}"),
        ]
```

```
476
477         for label, value in stats:
478             self.stat(x, y, label, value)
479             y += 45
480
481         status_color = TEXT
482         if status == "GAME OVER - CAUGHT":
483             status_color = DANGER
484         elif status == "VICTORY":
485             status_color = SUCCESS
486
487         self.stat(x, y, "GAME STATUS", status, status_color)
488         y += 55
489
490         self.screen.blit(self.small.render("CONTROLS", True, MUTED), (x, y))
491         y += 22
492
493         lines = [
494             "Arrow Keys : Move Pac-Man",
495             "[R] : Restart Game",
496             "[ESC] : Exit",
497         ]
498         for line in lines:
499             self.screen.blit(self.small.render(line, True, TEXT), (x, y))
500             y += 20
501
502         y += 8
503         self.screen.blit(
504             self.small.render("Green = A* chosen path", True, GREEN), (x, y)
505         )
506         y += 19
507         self.screen.blit(
508             self.small.render("Yellow = Minimax evaluated", True, SEARCH), (x, y
                )
509         )
510         y += 19
511         self.screen.blit(
512             self.small.render("Red = Alpha-Beta pruned", True, RED), (x, y)
513         )
514         y += 27
515         self.screen.blit(
516             self.font.render("Press [R] to Restart", True, ACCENT), (x, y)
517         )
518
519
520 # ================================================================================
521 # SECTION 6: MAIN EVENT LOOP & GAME CONTROLLER
522 # ================================================================================
523
524 class Game:
525     """Coordinates player input, AI updates, scoring, timing, and rendering."""
526
527     def __init__(self):
528         pygame.init()
```

```
        pygame.display.set_caption("Smart Pac-Man - AI Enemy Hunter")
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        self.clock = pygame.time.Clock()

        self.maze = Maze(MAZE_MAP)
        self.player = Player()
        self.ghost = GhostAI()
        self.coins = CoinManager(self.maze)
        self.renderer = Renderer(self.screen)

        self.score = 0
        self.status = "PLAYING"
        self.started_at = time.perf_counter()
        self.finished_at = None

    def restart(self):
        self.player.reset()
        self.ghost.reset()
        self.coins.reset()
        self.score = 0
        self.status = "PLAYING"
        self.started_at = time.perf_counter()
        self.finished_at = None
        print("[GAME] Restarted.")

    def elapsed(self):
        end_time = self.finished_at
        if end_time is None:
            end_time = time.perf_counter()
        return end_time - self.started_at

    def check_result(self):
        if self.player.position == self.ghost.position:
            self.status = "GAME OVER - CAUGHT"
            self.finished_at = time.perf_counter()
            print(f"[GAME] Pac-Man caught. Final Score: {self.score}")
            return

        if self.coins.remaining() == 0:
            self.status = "VICTORY"
            self.finished_at = time.perf_counter()
            print(f"[GAME] Victory! Final Score: {self.score}")

    def process_player(self):
        if self.status != "PLAYING":
            return

        keys = pygame.key.get_pressed()
        direction = None

        if keys[pygame.K_UP]:
            direction = "UP"
        elif keys[pygame.K_DOWN]:
            direction = "DOWN"
```

```
        elif keys[pygame.K_LEFT]:
            direction = "LEFT"
        elif keys[pygame.K_RIGHT]:
            direction = "RIGHT"

        if direction is None:
            return

        moved = self.player.move(
            self.maze, direction, time.perf_counter()
        )

        if moved:
            if self.coins.collect(self.player.position):
                self.score += 10
            self.check_result()

    def update_ai(self):
        if self.status != "PLAYING":
            return

        moved = self.ghost.update(
            self.maze,
            self.player.position,
            time.perf_counter(),
        )

        if moved:
            self.check_result()

    def run(self):
        running = True

        while running:
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    running = False

                elif event.type == pygame.KEYDOWN:
                    if event.key == pygame.K_ESCAPE:
                        running = False
                    elif event.key == pygame.K_r:
                        self.restart()

            self.process_player()
            self.update_ai()

            self.renderer.draw(
                self.maze,
                self.player,
                self.ghost,
                self.coins,
                self.score,
                self.elapsed(),
```

```
                self.status,
            )

            pygame.display.flip()
            self.clock.tick(FPS)

        pygame.quit()


def main():
    Game().run()


if __name__ == "__main__":
    main()
```